

relaleap_summary---36... PDF	SLT-accuracy-weaknes... PDF
Weakness-Singular-Lea... PDF	SLT-SubRep-v5.pdf PDF
columnar_residual_net... PDF	hibacaml_agi26(6).pdf PDF

The document "relaleap summary" describes some recent progress made along the "columnar residual" document's direction (see hibacaml paper for some background on aspects of the neural architecture, though the residual effort is focused on transformer NNs not vision). However progress has been slow and erratic, and has had too much trial and error aspect. I am wondering if singular learning theory (see other documents) can be helpful in figuring out a better and more principled way to drive the relaleap project forward? What does SLT say about how to make such a residual layer that disentangles the messy representation in a transformer NN?

Thought for 3m 45s >

According to a document from 2026-07-02, RelaLeap's current status is exactly the regime where SLT should help: contextual routing is a real positive result, but causal columnar separability is still unproven; flat value controls match or beat sparse arms on reconstruction; the orthogonal sparse-coding basis has been demoted to diagnostic status; and the next planned step is an alternative-basis pregate comparing non-orthogonal dictionaries, CE-gradient-aligned bases, and low-rank/SVD controls.

 relaleap_summary---3639dcbf-eac...

My main take: **SLT does not say "use this one basis" or "orthogonalize the hidden state."** It says: **treat the residual layer as a structure-learning problem over candidate factorizations, and promote a columnar decomposition only when it has high local evidence, low interaction remainder, stable causal fingerprints, and low commutator leakage.** In other words, "disentangling the messy transformer representation" should mean disentangling the **residual update geometry**, not necessarily the raw activation geometry.

The SLT criterion for a "real" residual column

A residual layer is genuinely disentangled when its excess loss locally factorizes:

$$K(\theta) \approx \sum_c K_c(\theta_c) + R(\theta),$$

where c indexes columns, K_c is the contribution of column c , and R is the interaction remainder. In SLT terms, exact factorization makes local evidence multiply, free energies add, and LLCs add. The Weakness/SLT document states this explicitly: if the evidence factorizes across modules, the global LLC is the sum of module LLCs. It also says that a single static factorization is too rigid, so one should maintain candidate factorizations in a refinement DAG and score them by weakness/evidence.

 Weakness-Singular-Learning

For RelaLeap, this means a column is not validated merely because the router selects it or because CE improves. It is validated if the trained residual around that column behaves like a **local factor** of the residual loss landscape. If the interaction term R is dominated by the factorized baseline, then the LLC does not change; if it is not dominated, the supposed columns are still entangled. The same Weakness/SLT appendix connects this directly to computable mixed-Hessian and commutator surrogates:

mixed Hessian blocks and out-of-support gradients are both commutator diagnostics and LLC-factorization diagnostics.  Weakness-Singular-Learning

So the first practical SLT rule is:

Do not promote a residual basis because it reconstructs well. Promote it because R is small relative to

That is why the current “flat value control beats sparse arms” result is important. It says the sparse basis has not yet earned its causal-factorization interpretation.  relaleap_summary---3639dcbf-eac...

What should replace trial-and-error?

Use an **SLT-informed pregate**. For every candidate residual basis f , score something like:

$$J(f) = n\hat{L}_f + \hat{\lambda}_f \log n - \log \pi(f) + \alpha \text{Comm}(f) + \beta \text{Leak}(f) + \gamma \text{Regret}(f) + \delta \text{Interaction}(f).$$

Here $\hat{L}_f$ is validation CE or residual reconstruction loss; $\hat{\lambda}_f$ is an estimated local learning coefficient; $\pi(f)$ is a prior favoring sparse/simple refinements; **Comm** is the $A \rightarrow B/B \rightarrow A$ finite-update commutator; **Leak** is off-support gradient leakage; **Regret** is oracle-support or one-swap support regret; and **Interaction** measures cross-column LLC or Hessian coupling. This is a direct translation of the Weakness/SLT view: weakness as local evidence, tropicalized into a min-plus refinement-DAG structure-selection problem.  Weakness-Singular-Learning

That changes the project from “try basis ideas until one works” to “compare candidate factorizations by a principled free-energy/evidence score plus causal-modularity constraints.”

What SLT suggests for the next RelaLeap basis pregate

The next pregate should not merely compare orthogonal sparse coding, learned dictionaries, CE-gradient bases, and SVD/low-rank controls on R^2 and CE. It should compare them on **five structure signals**.

First, estimate **global and module-wise LLC**. A basis is promising if the joint residual LLC is close to the sum of its module LLCs, after allowing for known shared structure. Large unexplained deviations mean the basis is not really decomposing the residual mechanism.

Second, measure **LLC interaction information**:

$$I_\lambda(A; B) = \lambda_A + \lambda_B - \lambda_{AB}.$$

The SubRep SLT document interprets $I_\lambda > 0$ as redundant/shared singular structure and $I_\lambda < 0$ as synergistic/emergent singular structure. For RelaLeap, positive interaction between two columns suggests “merge, share a core, or move common structure inward.” Strong negative interaction across supposedly separate columns suggests “these columns are not independently deployable; keep them in an assembly or introduce a mediator.”  SLT-SubRep-v5

Third, compute **mixed-Hessian / commutator dominance**. For columns a, b , estimate cross-Hessian blocks or Gauss-Newton/Fisher surrogates. If cross-block norms remain below within-block curvature in the domination sense, the factorization is stable; if they spike under distribution shift, the factorization has broken and the structure controller should split, merge, re-route, or add a mediator.

 Weakness-Singular-Learning

Fourth, use **causal fingerprints rather than activation clusters**. The columnar residual whitepaper argues that atom and column roles should be determined by ablation impact, not activation correlation, and that exact ablations should serve as calibration labels for information-geometric surrogates. It also notes that pairwise synergy/redundancy can be captured as the quadratic cross term in the local loss expansion.

 columnar_residual_networks_whit...  columnar_residual_networks_whit...

Fifth, track **support-selection regret**. A router can improve CE while still selecting columns for capacity rather than causal role. RelaLeap already saw that top-k=2 gains were confounded by active rank/capacity rather than proven cooperation. The pregate should therefore require low oracle-support regret and

strong one-swap/local support optimality before treating a router's support as evidence of modularity.

[relaleap_summary---3639dcfb-eac...](#)

The architecture SLT points toward

The residual layer should look less like "a sparse coding basis attached to hidden states" and more like a **Bayesian structure controller over residual mechanisms**:

$$h'_t = h_t + \sum_{c \in S_t} r_c(h_t; \theta_c),$$

where each r_c is a column with a protected core and adaptable periphery. The HBCML/HIBACAML direction is relevant here because it separates two processes: a sparse top-level controller selecting columns, and an internal process inside each column distinguishing reusable causal structure from task-local residue. [hibacaml_agi26](#)

For transformers, the Weakness/SLT document says natural modules include attention heads, MLP channel groups, and task-routing partitions; the goal is to use gates, clarity penalties, and LLC/weakness signals to maintain small supports, small commutators, and enough degeneracy for adaptation without collapsing the model into a brittle subspace. [Weakness-Singular-Learning](#)

Translated into RelaLeap terms:

Column core = *stable, repeatedly evidenced residual mechanism*;

Column periphery = *context – specific correction, easy to prune or reassign*;

Mediator column = *controlled bridge when two residual factors are repeatedly coupled*;

Router = *approximate posterior over which local factorization is active in this context*.

This is also consistent with the columnar residual whitepaper's master theorem: global weakness, commutator norm, forgetting, and interpretability are all controlled by the same structural quantities—atom weakness, intra-column redundancy, off-support leakage, cross-column coupling, and selector regret. [columnar_residual_networks_whit...](#)

A concrete SLT-driven RelaLeap experimental plan

For the next local pregate, I would run the following matched arms:

1. same-router flat value control;
2. SVD/low-rank dense control;
3. orthogonal sparse-coding diagnostic basis;
4. non-orthogonal learned dictionary;
5. CE-gradient-aligned dictionary;
6. Fisher/Gauss-Newton-aligned dictionary;
7. rank-one atom dictionary with on-center/off-surround column formation.

For each arm, log:

$$\hat{L}, \quad R^2, \quad \hat{\lambda}_{\text{joint}}, \quad \sum_c \hat{\lambda}_c, \quad I_\lambda(c, d), \quad \|H_{cd}\|, \quad \text{Comm}_{A,B}, \quad \text{Leak}, \quad \text{SupportRegret}, \quad \text{FingerprintSeparation}.$$

Then promote only an arm that passes all of these conditions:

$\hat{L}$ competitive with flat/SVD control,

$J(f) = n\hat{L} + \hat{\lambda} \log n - \log \pi(f)$ better than controls,

$\hat{\lambda}_{\text{joint}} \approx \sum_c \hat{\lambda}_c$ except for interpretable shared-core terms,

$I_\lambda(c, d)$ sparse and clustered, not diffuse,

$\|H_{cd}\|$ and finite-update commutators below budget,

singleton and pair causal efficacy pass matched controls,
support regret low under one-swap/oracle audits.

This would convert the current “basis pregate” into a real SLT/causal-factorization pregate.

What this says about “disentangling messy transformer representations”

The transformer’s hidden state may remain superposed and messy. SLT does not require that hidden coordinates become human-clean features. It requires that the **residual corrections** become locally factored in the evidence geometry. The residual columns are therefore best understood as **local charts on the transformer’s failure modes**, not as global semantic features.

So the right target is not:

“Find the true basis of the transformer representation.”

It is:

“Find a sparse, context-dependent residual factorization whose evidence decomposes, whose interaction remainder is dominated, whose causal fingerprints are separable, and whose update fields approximately commute.”

That is a much more operational target, and it directly explains why the current orthogonal basis can look good as an oracle diagnostic while failing as a deployable causal mechanism.

Bottom line

SLT can help RelaLeap most by changing the promotion criterion. The project should stop asking, “Which sparse residual basis reconstructs the dense teacher best?” and ask instead:

Which residual factorization has the best local evidence while keeping cross-column interactions c

That means choosing columns by evidence/free-energy, validating them by LLC additivity or interpretable LLC interactions, policing them by mixed-Hessian/commutator audits, and letting a refinement-DAG controller split, merge, mediate, or demote columns when the factorization fails. The result is a principled path from trial-and-error basis search toward a real causal modular residual layer.



📄 👍 🗨️ 📝 ⬆️ ↺ ... 📖 Sources

I like the above direction. Can you give more detaile dmathematicla and software suggestions for each of the 7 steps in the above epxerimental plan, suitable for guiding coding agents?

Thought for 6m 2s >

According to a document from 2026-07-02, RelaLeap’s immediate local next step is an **alternative residual-basis pregate** comparing non-orthogonal learned dictionaries, CE-gradient-aligned bases, and low-rank/SVD controls before GPU validation. The same document says the current central claim remains unproven because flat value controls match or beat sparse arms on residual reconstruction, finite-update commutator gates remain problematic, and the orthogonal sparse-coding basis has been demoted to diagnostic status. [relaleap_summary---3639dcbf-eac...](#) [relaleap_summary---3639dcbf-eac...](#)

I would treat the “seven steps” as **seven matched experimental arms in one controlled pregate**, not as a sequential pipeline. The goal is to discover whether any residual factorization earns a stronger claim than “a decent low-rank/dense correction with a good router.”

Common mathematical and software frame

All seven arms should implement the same interface and be evaluated on the same cached tensors.

Let the frozen transformer produce hidden states

$$h_i \in \mathbb{R}^d,$$

where i indexes batch-position examples. Let the dense teacher residual be

$$r_i^\star \in \mathbb{R}^d,$$

and let an arm f produce

$$\hat{r}_i^{(f)} = A_f(h_i, \phi_i; \theta_f),$$

where ϕ_i is the router/context feature vector. The adapted hidden state is

$$h'_i = h_i + \hat{r}_i^{(f)}.$$

The shared arm loss should be

$$\mathcal{L}_f = \text{CE}(y_i, \text{logits}(h'_i)) + \alpha_{\text{rec}} \|r_i^\star - \hat{r}_i^{(f)}\|_2^2 + \alpha_{\text{KL}} \text{KL}(p_{0,i} \| p_{f,i}) + \Omega_f.$$

The CE term keeps the arm honest with respect to the model's real task. The reconstruction term compares to the dense teacher residual. The KL term prevents broad collateral drift. Ω_f is arm-specific regularization.

The SLT-flavored selection score should be stored for every arm:

$$J(f) = n\widehat{\mathcal{L}}_f + \hat{\lambda}_f \log n - \log \pi(f) + \alpha C_f + \beta G_f + \gamma R_f + \delta I_f.$$

Here $\hat{\lambda}_f$ is a practical LLC proxy, C_f is commutator cost, G_f is off-support gradient or Hessian leakage, R_f is support regret, and I_f is interaction penalty or interaction structure score. This follows the SLT/weakness view that local evidence/free energy, not only point loss, should select among candidate factorizations. In the weakness/SLT document, weakness is identified with local evidence, and the LLC is the exponent governing its asymptotic volume term. [Weakness-Singular-Learning](#)

The practical LLC estimate should be explicitly labeled as a proxy, not as an exact RLCT. For each fitted arm, run a local SGLD/WBIC-style estimator around the trained solution:

$$p_\beta(\theta_f) \propto \exp[-\beta n \mathcal{L}_n(\theta_f)] \pi(\theta_f), \quad \beta = \frac{1}{\log n}.$$

Then estimate

$$\hat{\lambda}_f \approx \frac{\mathbb{E}_{\theta \sim p_\beta}[n \mathcal{L}_n(\theta)] - n \mathcal{L}_n(\hat{\theta}_f)}{\log n}.$$

This is motivated by the SLT free-energy expansion

$$F_n = n \ell_n(w^\star) + \lambda \log n - (m-1) \log \log n + O_p(1),$$

which appears in the uploaded SLT material. [SLT-SubRep-v5](#)

The coding agents should implement the shared interface like this:

```
class BasisArm:
    arm_id: str

    def fit(self, train_cache, val_cache, config) -> "BasisArm":
        ...

    def forward(self, h, context_features, return_aux=False):
```

```

# returns residual_hat, aux
...

def evaluate(self, eval_cache) -> dict:
    # CE, R2, KL drift, active rank, support stats
    ...

def audit(self, audit_cache) -> dict:
    # singleton gains, pair synergy, support regret,
    # commutator, leakage, LLC proxy, causal fingerprints
    ...

```

Every run should produce the existing RelaLeap-style artifacts:

```

runs/basis_pregate/<arm_id>/<seed>/
config.yaml
summary.json
metrics.csv
notes.md
decision.md
tensors/
  residual_basis.pt
  router_state.pt
  support_cache.pt
  llc_samples.pt
  audit_pack.pt

```



The RelaLeap harness already emphasizes command-driven experiments with `summary.json`, `metrics.csv`, and `notes.md`, plus fail-closed decision reports rather than ad hoc inspection.

relaleap_summary---3639dcbf-eac...

Shared diagnostics for all seven arms

Each arm should report the following.

Task and reconstruction metrics

$$\text{CE}_{\text{val}}, \quad R^2 = 1 - \frac{\sum_i \|r_i^* - \hat{r}_i\|^2}{\sum_i \|r_i^* - \bar{r}^*\|^2}, \quad \text{KL}_{\text{anchor}}.$$

Support regret

For a router-selected support S_i , compare against the best audited local support:

$$\text{Regret}_i = \mathcal{L}_i(S_i) - \min_{S \in \mathcal{N}(S_i)} \mathcal{L}_i(S).$$

Use exhaustive support search where possible; otherwise use one-swap or top- M candidate neighborhoods. The columnar residual whitepaper gives the same logic for surrogate audit regret: if the surrogate objective is uniformly accurate over the local candidate neighborhood, local support selection is within 2ζ regret of the audited optimum. columnar_residual_networks_whit...

Singleton gain and pair synergy

For a candidate column or basis atom a ,

$$G_a = \mathcal{L}(0) - \mathcal{L}(a).$$

For a pair,

$$\text{Syn}_{ab} = [\mathcal{L}(0) - \mathcal{L}(a, b)] - G_a - G_b.$$

Positive values mean the pair does more together than the sum of singleton effects. Negative values mean redundancy or interference.

Information-geometric pair interaction

For residual perturbations $\delta h_a, \delta h_b$,

$$\sigma_{ab} = \delta h_a^\top G_h \delta h_b,$$

where G_h is a Hessian, Fisher, Gauss–Newton, or learned metric. The columnar residual document states exactly this quadratic pairwise interaction form and explains that pairwise synergy/redundancy is the cross term of the local quadratic approximation, not an extra heuristic. [columnar_residual_networks_whit...](#)

Mixed-Hessian leakage

For column or basis blocks a, b ,

$$H_{ab} = P_a^\top H P_b, \quad \text{Leak}_{ab} = \frac{\|H_{ab}\|_F}{\sqrt{\|H_{aa}\|_F \|H_{bb}\|_F} + \epsilon}.$$

The Weakness/SLT appendix gives the useful domination principle: if the interaction remainder is dominated by the factorized baseline, the LLC does not change; in a quadratic regime, sufficiently small mixed-Hessian blocks provide a computable domination test. [Weakness-Singular-Learning](#)

LLC interaction information

For modules a, b ,

$$I_\lambda(a; b) = \hat{\lambda}_a + \hat{\lambda}_b - \hat{\lambda}_{ab}.$$

Use this as a structural diagnostic:

$$\begin{aligned} I_\lambda > 0 &\Rightarrow \text{redundant/shared singular structure,} \\ I_\lambda < 0 &\Rightarrow \text{synergistic/emergent singular structure.} \end{aligned}$$

The SubRep SLT document defines this LLC interaction complexity and interprets its sign in precisely this way. [SLT-SubRep-v5](#)

Finite-update commutator

For two microtasks or sequence slices A, B , train tiny updates in both orders:

$$\theta_{A \rightarrow B} = U_B(U_A(\theta_0)), \quad \theta_{B \rightarrow A} = U_A(U_B(\theta_0)).$$

Measure

$$\text{Comm}(A, B) = \mathbb{E}_{x \in \mathcal{D}_{\text{audit}}} \|\text{logits}_{\theta_{A \rightarrow B}}(x) - \text{logits}_{\theta_{B \rightarrow A}}(x)\|_2^2.$$

This should remain a first-class gate because Relaleap's promoted top- $k = 2$ arm failed the finite-update commutator budget, and several mitigation families failed their own gates. [relaleap_summary---3639dcbf-eac...](#)

Arm 1: same-router flat value control

Mathematical specification

This arm asks: **Does the router already explain the gain, without any meaningful basis structure?**

Use the same router scores $s_c(h_i)$ and support S_i as the sparse arms, but give every selected column a free dense value vector:

$$\hat{r}_i = \sum_{c \in S_i} \alpha_{ic} v_c, \quad \alpha_{ic} = \text{softmax}_{c \in S_i}(s_c(h_i)).$$

Here

$$v_c \in \mathbb{R}^d$$

is unconstrained except for optional norm control. This is deliberately simple. It tests whether "column identity + router context" is enough.

Use a matched version too:

$$v_c = Bu_c, \quad B \in \mathbb{R}^{d \times q},$$

where q is chosen so the active rank matches the sparse arms. This prevents the flat control from winning merely because it has more effective rank.

Loss

$$\mathcal{L}_{\text{flat}} = \text{CE} + \alpha_{\text{rec}} \|r^* - \hat{r}\|^2 + \alpha_v \sum_c \|v_c\|^2 + \alpha_{\text{KL}} \text{KL}(p_0 \| p_{\text{flat}}).$$

Software tasks

Create:

```
relaleap/arms/flat_value.py
```



Implement:

```
class FlatValueControl(BasisArm):
    def __init__(self, router, num_columns, hidden_dim, rank=None):
        ...
```

Support two modes:

```
arm_id: flat_value_same_router
router_source: contextual_mlp_topk2
top_k: 1 or 2
rank_match: null           # unconstrained dense values
rank_match: 4             # low-rank value subspace
freeze_router: true
train_values: true
```

The `forward` method should return:

```
{
    "residual": residual_hat,
    "support": support_indices,
    "support_weights": alpha,
    "active_rank": estimated_rank,
    "value_norm": value_norm
}
```

Required audits

1. Same-context CE and R^2 .
2. Active-rank-adjusted CE.
3. Singleton gain for each selected c .
4. Pair synergy for top- $k = 2$.
5. Commutator $A \rightarrow B / B \rightarrow A$.
6. Support regret against one-swap support.

Decision rule

This arm is a **floor**, not a candidate mechanism. If a proposed sparse/basis arm does not beat flat value control on

$$J(f),$$

or at least beat it on commutator/leakage/causal interpretability while matching CE, then the sparse arm should not be promoted.

RelaLeap already found that same-router flat value control exceeded the sparse arm on R^2 and oracle-gain retention, so this control is essential.  relaLeap_summary---3639dcbf-eac...

Arm 2: low-rank / SVD dense control

Mathematical specification

This arm asks: **Is the dense teacher residual merely low-rank, rather than columnar?**

Let

$$R^* = \begin{bmatrix} (r_1^*)^\top \\ \vdots \\ (r_N^*)^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

Compute truncated SVD:

$$R^* \approx U_q \Sigma_q V_q^\top.$$

Let

$$D_{\text{SVD}} = V_q \in \mathbb{R}^{d \times q}.$$

Use either oracle coefficients

$$z_i^* = D_{\text{SVD}}^\top r_i^*, \quad \hat{r}_i = D_{\text{SVD}} z_i^*,$$

or deployable coefficients

$$z_i = g_\theta(h_i, \phi_i), \quad \hat{r}_i = D_{\text{SVD}} z_i.$$

The oracle version gives an upper bound. The deployable version gives a fair baseline.

A useful variant is Fisher-weighted SVD:

$$\min_{D, Z} \sum_i (r_i^* - D z_i)^\top G_i (r_i^* - D z_i),$$

but keep ordinary SVD first because it is easier to debug.

Loss

For deployable coefficients:

$$\mathcal{L}_{\text{SVD}} = \text{CE} + \alpha_{\text{rec}} \|r^* - D_{\text{SVD}} g_\theta(h, \phi)\|^2 + \alpha_z \|z\|^2 + \alpha_{\text{KL}} \text{KL}(p_0 \| p_f).$$

Software tasks

Create:

```
relaLeap/arms/svd_lowrank.py
relaLeap/basis/svd.py
```



Implement:

```
def fit_truncated_svd(residual_matrix, q, center=True):
    # returns D, singular_values, explained_variance
```

and:

```
class LowRankDenseControl(BasisArm):
    def __init__(self, basis_D, coeff_net):
        ...
```

Config:

```
arm_id: svd_lowrank_q4
basis:
  type: svd
  q: 4
  center_residuals: true
coefficients:
  mode: oracle | linear | mlp
rank_match_to_sparse: true
```

Required audits

- 1. Explained residual variance by rank:

$$EV(q) = \frac{\sum_{j \leq q} \sigma_j^2}{\sum_j \sigma_j^2}.$$

- 2. CE vs rank curve:

$$q \in \{1, 2, 4, 8, 16\}.$$

- 3. Active rank vs CE.
- 4. Anchor KL drift.
- 5. Commutator.

Decision rule

If SVD rank q matches or beats a sparse arm with lower commutator and simpler implementation, then the result is **low-rank residual adaptation**, not evidence for columnar separability.

If sparse arms beat SVD only at larger active rank, treat that as capacity, not modularity.

Arm 3: orthogonal sparse-coding diagnostic basis

Mathematical specification

This arm asks: What is the best sparse approximation ceiling under an orthogonal diagnostic basis?

Let

$$D \in \mathbb{R}^{d \times m}, \quad D^\top D = I.$$

For oracle sparse coding,

$$\begin{aligned} u_i &= D^\top r_i^\star, \\ S_i^{\text{oracle}} &= \text{TopK}(|u_i|, k), \\ \hat{r}_i^{\text{oracle}} &= D_{S_i} u_{i, S_i}. \end{aligned}$$

Then compare four crossed variants:

Variant	Support	Coefficients
O/O	oracle	oracle
R/O	router	oracle

Variant	Support	Coefficients
O/L	oracle	learned
R/L	router	learned

The key decomposition is:

$$\text{oracle gap} = \mathcal{L}_{R/L} - \mathcal{L}_{O/O}.$$

Then split it into support error and coefficient error:

$$\text{support gap} = \mathcal{L}_{R/O} - \mathcal{L}_{O/O},$$

$$\text{coefficient gap} = \mathcal{L}_{O/L} - \mathcal{L}_{O/O}.$$

Software tasks

Create:

```
relaleap/arms/orthogonal_sparse_diag.py
relaleap/audits/oracle_sparse.py
```



Implement:

```
def oracle_topk_orthogonal(D, residuals, k):
    u = residuals @ D
    support = topk(abs(u), k)
    coeffs = gather(u, support)
    recon = scatter_to_basis(D, support, coeffs)
    return recon, support, coeffs
```

Add crossed-mode evaluation:

```
arm_id: orthogonal_sparse_diag
basis:
  type: orthogonal
  source: pca | random_orthogonal | previous_sparse_coding
top_k: 1 or 2
modes:
  - oracle_support_oracle_coeff
  - router_support_oracle_coeff
  - oracle_support_learned_coeff
  - router_support_learned_coeff
```

Required audits

1. R^2 oracle ceiling.
2. CE oracle ceiling.
3. Oracle-gain retention:

$$\text{Retention} = \frac{\text{Gain}_{\text{deployable}}}{\text{Gain}_{\text{oracle}} + \epsilon}.$$

4. Support regret.
5. Frequency-matched random support null.
6. Learned-router vs oracle-support confusion matrix.

Decision rule

This arm should **not** be promoted as mechanism unless it unexpectedly beats flat/SVD deployably and passes causal gates. Its main use is diagnostic: it tells whether a sparse support could work in principle.

This matches Relaleap's current conclusion that the orthogonal sparse-coding basis is a useful oracle ceiling but not yet a deployable mechanism.  relaleap_summary---3639dcbf-eac...

Arm 4: non-orthogonal learned dictionary

Mathematical specification

This arm asks: **Are the causal residual factors oblique rather than orthogonal?**

Use a learned dictionary

$$D = [d_1, \dots, d_m] \in \mathbb{R}^{d \times m}, \quad \|d_j\|_2 = 1,$$

with sparse coefficients

$$z_i = E_\theta(h_i, \phi_i), \quad \|z_i\|_0 \leq k,$$

and

$$\hat{r}_i = Dz_i.$$

The training objective is

$$\mathcal{L}_{\text{dict}} = \text{CE} + \alpha_{\text{rec}} \|r_i^* - Dz_i\|^2 + \lambda_1 \|z_i\|_1 + \lambda_{\text{usage}} L_{\text{usage}} + \lambda_{\text{coh}} L_{\text{coh}} + \lambda_{\text{KL}} \text{KL}(p_0 \| p_f).$$

Use a **soft incoherence** penalty, not a hard orthogonality penalty:

$$L_{\text{coh}} = \sum_{a \neq b} w_{ab} (d_a^\top d_b)^2.$$

Set w_{ab} higher when atoms co-activate or have similar causal fingerprints. Do not globally force all dictionary atoms apart; causal mechanisms can be oblique or share substructure.

A useful SLT-motivated quantity is the interaction remainder:

$$K(\theta) = \sum_c K_c(\theta_c) + R(\theta).$$

The goal is not $D^\top D \approx I$. The goal is

$$|R(\theta)| \leq \eta \sum_c K_c(\theta_c), \quad \eta < 1,$$

because dominated interaction preserves LLC additivity. The Weakness/SLT document gives this dominated-interaction criterion explicitly.  Weakness-Singular-Learning

Software tasks

Create:

```
relaleap/arms/nonorth_dictionary.py
relaleap/basis/dictionary_learning.py
```



Implement three coefficient modes:

```
class NonOrthogonalDictionaryArm(BasisArm):
    coeff_mode: "oracle_omp" | "lista_encoder" | "router_topk"
```

Oracle OMP:

```
def omp(D, residual, k):
    # greedy least-squares sparse coding
```

Deployable encoder:

```
z_logits = encoder(torch.cat([h, context_features], dim=-1))
support = topk(z_logits, k)
z = masked_coefficients(z_logits, support)
r_hat = D @ z
```

Use straight-through top- k , entmax, or sparsemax. Log which relaxation is used.

Config:

```
arm_id: nonorth_dict_m64_k2
dictionary:
  m: 64
  normalize_atoms: true
  init: svd | random | grad_pca
coefficients:
  mode: lista_encoder
  top_k: 2
regularizers:
  l1_coeff: 1e-4
  usage_entropy: 1e-3
  coactivation_incoherence: 1e-3
```

Required audits

1. Dictionary collapse:

$$\max_{a \neq b} |d_a^\top d_b|.$$

2. Dead atom fraction:

$$\frac{|\{j : \text{usage}(j) < \tau\}|}{m}.$$

3. Co-activation-weighted coherence.
4. Support regret vs OMP oracle.
5. Mixed-Hessian block norms between active dictionary groups.
6. Pairwise $I_\lambda(a; b)$ for frequently co-used atoms.

Decision rule

Promote only if it beats SVD/flat on the full score $J(f)$, not only on reconstruction. A non-orthogonal dictionary that improves R^2 but creates dense I_λ , high commutator, or diffuse mixed-Hessian leakage is just a better compressor, not a columnar residual mechanism.

Arm 5: CE-gradient-aligned dictionary

Mathematical specification

This arm asks: **Should basis directions be chosen by task loss sensitivity rather than residual variance?**

For each hidden state, compute

$$g_i = \nabla_{h_i} \text{CE}(y_i, \text{logits}(h_i)).$$

A small residual δh changes CE approximately as

$$\Delta \mathcal{L}_i \approx g_i^\top \delta h + \frac{1}{2} \delta h^\top H_i \delta h.$$

So useful first-order correction directions are aligned with $-g_i$. Build a dictionary from the empirical gradient matrix

$$G = \begin{bmatrix} g_1^\top \\ \vdots \\ g_N^\top \end{bmatrix}.$$

Options:

Gradient PCA

$$G \approx \tilde{U}_q \tilde{\Sigma}_q \tilde{V}_q^\top, \quad D_{\text{grad}} = \tilde{V}_q.$$

Residual-gradient CCA

Find directions d with high covariance between teacher residual and CE gradient:

$$\max_d \frac{d^\top C_{rg} d}{\sqrt{d^\top C_{rr} d} \sqrt{d^\top C_{gg} d}}.$$

Supervised CE dictionary

Train D , z_i directly to reduce CE:

$$\min_{D, E} \text{CE}(y_i, \text{logits}(h_i + DE(h_i, \phi_i))) + \lambda \|z_i\|_1.$$

This arm may have lower R^2 than SVD but better CE. That is acceptable. Record both.

Software tasks

Create:

```
relaleap/arms/ce_gradient_dictionary.py
relaleap/cache/gradient_cache.py
```



Implement:

```
def build_hidden_gradient_cache(model, dataloader, insertion_site):
    # stores h_i, y_i, logits_i, grad_h_i, optional residual_teacher_i
```

Basis builders:

```
def gradient_pca_basis(grad_matrix, q):
    ...

def residual_gradient_cca_basis(residual_matrix, grad_matrix, q):
    ...
```

Adapter:

```
class CEGradientDictionaryArm(BasisArm):
    basis_type: "grad_pca" | "residual_grad_cca" | "supervised"
```

Config:

```
arm_id: ce_grad_pca_q16_k2
basis:
```

```

type: grad_pca
q: 16
coefficients:
  mode: learned_encoder
  top_k: 2
loss_weights:
  ce: 1.0
  reconstruction: 0.2
  anchor_kl: 0.05

```

Required audits

1. CE gain per residual norm:

$$\frac{\Delta \text{CE}}{\mathbb{E} \|\hat{r}\|^2 + \epsilon}.$$

2. R^2 against dense teacher.
3. Anchor KL drift.
4. Sign correctness:

$$\Pr[g_i^\top \hat{r}_i < 0].$$

5. Singleton causal efficacy.
6. Off-context forced singleton interference.

The last two matter because RelaLeap found context-gated singleton efficacy but negative singleton gain in isolation. [relaleap_summary---3639dcbf-eac...](#)

Decision rule

A CE-gradient basis should not be rejected merely because it reconstructs less teacher residual than SVD. It should be rejected if its CE gains come with high KL drift, poor support stability, high commutator, or failure on exact causal audits.

Arm 6: Fisher / Gauss–Newton aligned dictionary

Mathematical specification

This arm asks: **Should the residual basis be aligned with the model's local information geometry?**

For logits z , probabilities p , and hidden-to-logit Jacobian J_i ,

$$G_i = J_i^\top [\text{diag}(p_i) - p_i p_i^\top] J_i$$

is the hidden-space Fisher/Gauss–Newton metric. Aggregate:

$$G = \frac{1}{N} \sum_i G_i.$$

There are two important basis constructions.

Fisher eigenbasis

$$Gv_j = \lambda_j v_j.$$

This finds sensitive directions.

Residual-over-Fisher generalized eigenbasis

Let

$$C_R = \frac{1}{N} \sum_i r_i^* (r_i^*)^\top.$$

Solve

$$C_R v = \rho(G + \epsilon I) v.$$

This finds directions with high residual energy but controlled Fisher cost. I would prioritize this over raw Fisher eigenvectors, because high-Fisher directions may be powerful but brittle.

A Fisher-whitened sparse coding version is also useful:

$$\tilde{r}_i = (G + \epsilon I)^{1/2} r_i^*,$$

learn sparse codes in whitened space, then map back:

$$\hat{r}_i = (G + \epsilon I)^{-1/2} D \tilde{z}_i.$$

Software tasks

Create:

```
relaleap/arms/fisher_gn_dictionary.py
relaleap/metrics/fisher.py
relaleap/metrics/hvp.py
```



Implement low-rank metric estimation:

```
def estimate_gauss_newton_factor(model, cache, rank, damping):
    # returns R, diag_d such that G ≈ R.T @ R + diag(diag_d)
```

Use randomized HVP / JVP / VJP, not dense Hessian construction.

Basis builders:

```
def fisher_eigenbasis(metric_factor, q):
    ...

def residual_over_fisher_basis(residuals, metric_factor, q, damping):
    ...
```

Config:

```
arm_id: fisher_residual_gen_eig_q16_k2
metric:
  type: gauss_newton
  rank: 32
  damping: 1e-3
basis:
  type: residual_over_fisher
  q: 16
coefficients:
  mode: learned_encoder
  top_k: 2
```

Required audits

1. Fisher cost:

$$\mathbb{E}[\hat{r}_i^\top G_i \hat{r}_i].$$

2. CE gain per Fisher cost.
3. Pair interaction matrix:

$$\sigma_{ab} = \delta h_a^\top G \delta h_b.$$

4. KL drift.
5. Mixed-Hessian leakage.

6. Commutator $A \rightarrow B/B \rightarrow A$.

Decision rule

This arm is strong if it shows a sparse interaction matrix σ_{ab} , low KL drift, and low commutator while preserving CE. If it improves CE by moving in high-Fisher directions but produces broad anchor KL drift, reject it.

This arm also supplies infrastructure for the later surrogate audits. The columnar residual whitepaper specifically recommends Fisher/Gauss–Newton, low-rank curvature approximations, and learned interventional Jacobians as scalable replacements for exhaustive ablation. [columnar_residual_networks_whit...](#)

Arm 7: rank-one atom dictionary with on-center/off-surround column formation

Mathematical specification

This is the arm closest to the full columnar residual vision.

Each atom is a rank-one residual motif:

$$A_g(h) = s_g b_g (a_g^\top h),$$

where

$$a_g, b_g \in \mathbb{R}^d.$$

A column is a sparse coalition of atoms:

$$C_c(h) = \sum_{g \in G_c} q_{cg} z_g(h) A_g(h).$$

The active residual is

$$\hat{r}(h) = \sum_{c \in S(h)} \alpha_c(h) C_c(h).$$

This matches the columnar residual whitepaper's architecture, where rank-one atoms are organized into virtual columns and sparse supports. [columnar_residual_networks_whit...](#)

Each atom gets a functional signature:

$$\phi_g = [\phi_g^{\text{act}}, \phi_g^{\text{grad}}, \phi_g^{\text{abl}}, \phi_g^{\text{sub}}, \phi_g^{\text{layer}}, \phi_g^{\text{usage}}].$$

Column prototypes m_c live in the same signature space. Membership is updated by on-center/off-surround dynamics:

$$q_{cg} \propto \exp \left(-\frac{d(\phi_g, m_c)^2}{\tau} - \lambda_{\text{sur}} \sum_{b \neq c} K(c, b) q_{bg} \right).$$

The columnar whitepaper gives this exact form for prototype attraction plus surround inhibition.

[columnar_residual_networks_whit...](#)

Rank-one overlap is cheap:

$$O_{gh}^{\text{read}} = \cos^2(a_g, a_h),$$

$$O_{gh}^{\text{write}} = \cos^2(b_g, b_h),$$

$$O_{gh}^{\text{func}} = O_{gh}^{\text{read}} O_{gh}^{\text{write}}.$$

Redundancy weakness can be

$$w_g^{\text{redund}} = \sum_{h \neq g} O_{gh}^{\text{func}} |z_g| |z_h|.$$

The whitepaper also defines an atom weakness vector including forgetting, redundancy, misplacement, uncertainty, and related components.  columnar_residual_networks_whit...

Loss

$$\mathcal{L}_{\text{atom}} = \text{CE} + \alpha_{\text{rec}} \|r^* - \hat{r}\|^2 + \lambda_{\text{weak}} \sum_g \|w_g\| + \lambda_{\text{sur}} L_{\text{surround}} + \lambda_{\text{sparse}} L_{\text{sparse}} + \lambda_{\text{KL}} \text{KL}(p_0 \| p_f).$$

Use zero-initialized write vectors:

$$b_g = 0$$

so the adapter is identity at initialization.

Software tasks

Create:

```
relaleap/arms/rank_one_atoms.py
relaleap/columns/prototypes.py
relaleap/columns/weakness.py
relaleap/columns/structural_actions.py
```



Implement:

```
class RankOneAtom(nn.Module):
    a: Parameter # read vector
    b: Parameter # write vector, zero-init
    s: Parameter # scalar scale

    def forward(self, h):
        return self.s * self.b * (h @ self.a)
```

Column:

```
class AtomColumn(nn.Module):
    atom_ids: list[int]
    memberships: Tensor # q_cg

    def forward(self, h, atom_outputs, atom_gates):
        ...
```

Prototype updater:

```
def update_memberships(signatures, prototypes, q_prev, tau, lambda_sur, kernel):
    # implements on-center/off-surround soft assignment
```

Weakness metrics:

```
def rank_one_overlap(a, b):
    read = cosine_sq(a_i, a_j)
    write = cosine_sq(b_i, b_j)
    return read * write
```

```
def redundancy_weakness(atom_outputs, overlaps, gates):
    ...
```

Structural actions:

```
def propose_split(column):
    ...

def propose_merge(column_a, column_b):
    ...

def propose_prune(atom):
    ...

def accept_action(delta_weakness, audit_safety, threshold):
    return delta_weakness > threshold and audit_safety
```

Config:

```
arm_id: rank_one_atoms_columns
atoms:
  num_atoms: 128
  zero_init_write: true
  attach_site: residual_adapter
columns:
  num_columns: 32
  atoms_per_column: 8
  top_k_columns: 2
  membership: soft
prototype:
  tau: 0.1
  lambda_surround: 0.05
weakness:
  redundancy: 1e-3
  forgetting: 1e-2
  uncertainty: 1e-4
training:
  stages:
    - atom_warmup
    - router_warmup
    - joint_training
    - audit_checkpoint
    - structural_update
```

Required audits

1. Atom overlap matrix.
2. Column causal fingerprints.
3. Exact sampled atom ablations.
4. Surrogate ablation prediction error.
5. Column singleton gain.
6. Column pair synergy.
7. Router support entropy.
8. Dead-column and overused-column rates.
9. Commutator.
10. LLC additivity / interaction information by column.

Decision rule

This arm earns promotion only if it beats non-column controls on **causal-modularity metrics**, not merely CE.

Promote if:

$$J_{\text{atom-column}} < J_{\text{flat}}, \quad J_{\text{atom-column}} < J_{\text{SVD}},$$

and

Comm, Leak, SupportRegret

are all lower or statistically tied while interpretability metrics are better.

Reject or demote if router collapse occurs, if surround inhibition over-fragments atoms, or if exact ablations fail to confirm surrogate causal fingerprints. The columnar whitepaper explicitly lists these as major risks: routing collapse, over-fragmentation, noisy audit proxies, and miscalibrated weakness rewards.  [columnar_residual_networks_whit...](#)

Common coding-agent implementation plan

Ticket 1: build the cache layer

Create:

relaleap/cache/pregate_cache.py



It should cache:

```
PregateCache:
    h: FloatTensor           # [N, d]
    context_features: Tensor  # [N, cdim]
    targets: LongTensor      # [N]
    base_logits: FloatTensor  # [N, vocab]
    teacher_residual: FloatTensor # [N, d]
    token_ids: LongTensor
    batch_id: LongTensor
    position_id: LongTensor
```

Optional:

```
grad_h: FloatTensor
fisher_diag: FloatTensor
gn_lowrank: FloatTensor
```

Hard invariants:

```
assert_base_frozen()
assert_adapter_identity_at_init()
assert_same_examples_across_arms()
assert_same_router_when_config_requires_same_router()
```

Ticket 2: implement metrics registry

Create:

relaleap/metrics/pregate_metrics.py



Implement:

```

compute_ce()
compute_r2()
compute_anchor_kl()
estimate_active_rank()
compute_support_entropy()
compute_singleton_gain()
compute_pair_synergy()
compute_support_regret()
compute_commutator()
compute_hessian_leakage()
estimate_llc_proxy()
compute_llc_interaction()

```

Every metric should return:

```

{
  "mean": float,
  "stderr": float,
  "bootstrap_ci95": [low, high],
  "n": int
}

```

Ticket 3: implement fair matching

Create:

relaleap/controls/matching.py



Every arm must report:

```

num_trainable_params
num_active_params_per_token
active_rank_estimate
router_trainable_params
residual_norm_mean
residual_norm_p95

```

Add matching utilities:

```

match_active_rank(reference_arm, candidate_arm)
match_param_count(reference_arm, candidate_arm)
match_residual_norm(reference_arm, candidate_arm)

```

This matters because RelaLeap already found top- $k = 2$ gains may reflect active-rank capacity rather than cooperation. relaleap_summary---3639dcbf-eac...

Ticket 4: implement local LLC proxy

Create:

relaleap/slt/llc_proxy.py



Use SGLD around the fitted residual parameters only. Base model frozen. Usually freeze the router for module-wise LLC unless the specific test is "router + basis jointly."

Output:

For module-wise LLC:

For joint LLC:

Then:

Ticket 5: implement decision reports

Ticket 5: implement decision reports

Create:

Output:

The decision report should explicitly compare each arm against:

Promotion gates for the whole pregate

A basis arm should move forward only if all of the following hold.

First, it must be competitive on ordinary metrics:

$$\text{CE}_f \leq \text{CE}_{\text{flat}} + \epsilon_{\text{CE}}$$

and

$$R_f^2 \geq R_{\text{flat}}^2 - \epsilon_{R^2},$$

unless the arm is explicitly CE-first, like the CE-gradient arm.

Second, it must improve the SLT/evidence score:

$$J(f) < J(\text{flat})$$

under matched active rank and matched parameter budget.

Third, it must improve or tie causal metrics:

$$\text{Comm}_f \leq \text{Comm}_{\text{flat}},$$

$$\text{Leak}_f \leq \text{Leak}_{\text{flat}},$$

$$\text{SupportRegret}_f \leq \text{SupportRegret}_{\text{flat}}.$$

Fourth, its LLC interaction structure should be sparse and interpretable:

$$I_\lambda(a; b)$$

should cluster into a small number of redundant or synergistic pairs, not become diffuse across nearly every pair.

Fifth, exact ablations must confirm the surrogate audits on the local neighborhoods the controller searches:

$$|J_{\text{surrogate}}(S) - J_{\text{exact}}(S)| \leq \zeta$$

for all audited $S \in \mathcal{N}(S_0)$. The columnar whitepaper argues that exact ablations should teach and calibrate, while information geometry should handle routine causal auditing at scale.

 columnar_residual_networks_whit...

Most likely outcomes and what to do

If **flat value control wins**, the current router/context machinery is doing the work, and columnar claims should remain blocked.

If **SVD wins**, the residual is low-rank but not yet modular. The next step should be rank-matched low-rank adapters, not columns.

If **orthogonal sparse oracle wins but deployable sparse fails**, the problem is support prediction or coefficient prediction, not basis capacity.

If **non-orthogonal dictionary wins**, the next round should explore oblique causal factors and mixed-Hessian domination, not orthogonal sparse coding.

If **CE-gradient wins**, the residual target should be revised. The dense teacher residual may be the wrong teacher; CE-relevant residual geometry is more important than reconstructing the teacher.

If **Fisher/GN wins**, the project should pivot toward information-geometric bases and surrogate-audit infrastructure.

If **rank-one atoms win**, then RelaLeap has real evidence for the original columnar residual direction: not just sparse reconstruction, but causal, auditable, low-commutator modular residual adaptation.

